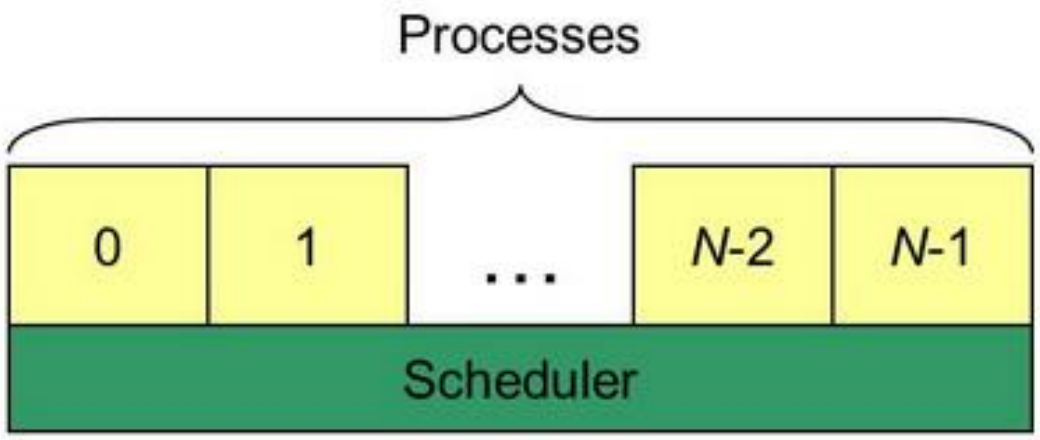
İşletim Sistemleri
(Operating Systems)

Process Zamanlama
(Process Scheduling)

Ardışık Process Yürütümü (Sequential Execution)



- Bir process temel olarak çalıştırılmakta olan bir programdır.
- Tek process'li işletim sistemlerinde process'ler ardışık yürütüm ile sırayla bir birleri ardından yürütülür.
- Ana bellekte, sadece yürütülmekte olan process ile ilgili bilgiler tutulur.

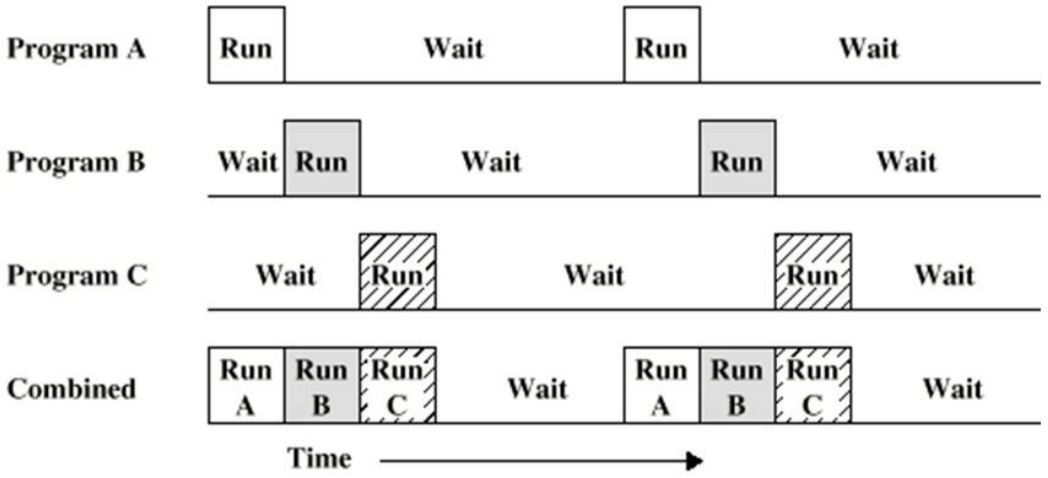
Eşzamanlı Process Yürütümü (Concurrent Execution)

- Bir bilgisayar sisteminde paylaşılan kaynaklar arasında en önemli kaynak CPU'dur.
- Çok kullanıcılı (multi-user) ve çok programlı (multi-programming) sistemlerde, aynı anda birden çok process yürütülmesi zorunluluğu, CPU'nun bu processler arasında paylaştırılmasını gerektirir.
- Çoklu programlamada bellekte birden fazla process yürütülmeye hazır beklerken, bu process'ler hangi sırayla çalıştırılacak?

Eşzamanlı Process Yürütümü (Concurrent Execution)

- Eğer bir işletim sistemi çok programlı (multi programming) desteğe sahip ise, sıklıkla birçok process aynı anda işlemciyi kullanmak için birbirleri ile yarışırlar.

- Bu durum aynı anda birden fazla process'in hazır (ready) durumunda bulunmasıyla gerçekleşir.



Process Zamanlama (Process Scheduling)

- İşletim sisteminde, birden fazla hazır durumda bulunan process'lerden hangisinin işlemciyi kullanacağına karar verilmelidir:

- Bu durum için **zamanlayıcı/iş sıralayıcı (scheduler)** kullanılır.

- Bu sistemde kullanılan algoritmaya da **zamanlama algoritması (scheduling algorithm)** denilir.

Process Zamanlama (Process Scheduling)

- Çok programlı sistemlerin amacı eşzamanlı yürütülen (concurrent execution) process sayısını artırarak CPU'yu maksimum etkinliğe ve üretkenliğe kavuşturmaktır.

- Tek işlemcili bir sistemde bir anda sadece bir process yürütülebilir. Birden fazla process olduğunda, bir process CPU'da yürütülürken diğerleri CPU'nun işinin bitmesi ve sıranın kendilerine gelmesi için Hazır (ready) kuyruğunda bekleyeceklerdir.

Process Zamanlama (Process Scheduling)

- Process'lerin **eşzamanlı (concurrent)** olarak CPU üzerinde yürütülebilmeleri için zamanlamalarının sağlanması gerekir.

- CPU zamanlama algoritmalarına göre sırası gelen process bekleme sırasından (Hazır Kuyruk) alınarak görevlendirici (dispatcher) tarafından CPU'ya gönderilir.

Process Zamanlama (Process Scheduling)

- CPU’nun bir process yürütmesini, sonradan tekrar devam etmek üzere bırakıp diğer bir process yürütmesine geçmesi process anahtarlama (process switching) olarak adlandırılır.

- Process’lerin hangi sıra ve kurallar çerçevesinde CPU üzerinde anahtarlanacağını işletim sistemi process yönetimi (process management) düzenler.

Process Zamanlama (Process Scheduling)

- CPU'nun, eşzamanlı yürütümü sağlamak üzere bir process yürütmesini bırakıp diğer bir process yürütmesine geçmesi belirli önlemler alınmadan yapılamaz, process yürütmesinin yarım bırakıldığı andaki process bağlam bilgilerinin saklanması gerekir (context switching).
- Bu nedenle, bağlam bilgisini tutmak için her process için işletim sistemi tarafından bir veri yapısı tutulur (process control block).
- Process'ler arasında bağlam anahtarlama (context switching) son derece maliyetli bir işlemdir.

Process Zamanlama (Scheduling)

- Zamanlayıcı (Scheduler), bir process'i işlemcide yürütülmesi için seçer ve böylece process'in durumu hazırdan (ready), koşuyor (running) durumuna geçer.

- Zamanlayıcı (Scheduler), daha sonra hazır durumda bulunan başka bir process'i seçer ve CPU'daki process'in yürütülmesi bittiğinde yeni process'in işlemcide yürütülmesini sağlar.

Process Zamanlama (Scheduling)

- Zamanlayıcı CPU'yu çok iyi kullanmalıdır ve CPU'dan maksimum performans alınmasını sağlamalıdır.
- Bu yüzden CPU mümkün olduğunca meşgul tutulmalıdır.

- Birim zamanda yürütülen process sayısını arttırmak önemlidir.
- Bir process'in yürütümünün başlangıcından bitimine kadar geçen toplam bitirilme zamanı asgari sürede tutulmalıdır.

Process Zamanlama (Scheduling)

- Process zamanlama açısından, process'lerin çalışma zamanındaki davranışları iki farklı kip ile modellenebilir :
- i. CPU'yu yoğun olarak kullanan process'ler (CPU bound)
- ii. Yoğun Giriş/Çıkış işleminde bulunan process'ler (I/O bound)

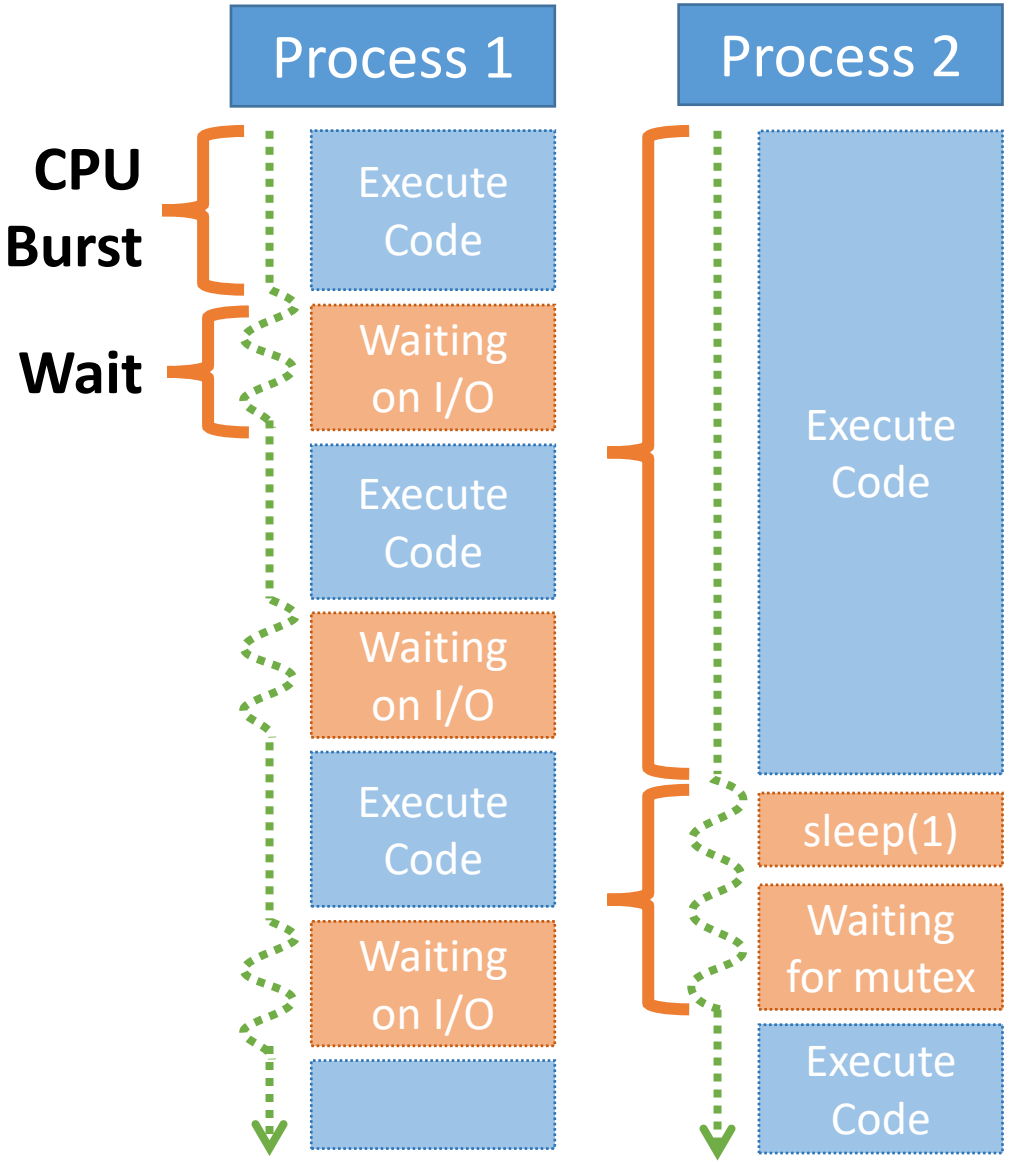
- Genel olarak, bir process'in çalışma zamanındaki davranışı bu iki kip arasında gidip gelecektir.

Process Zamanlama (Scheduling)

- Process'lerin çalışma zamanındaki CPU ve I/O sistemini kullanım davranışı değişir.

- I/O bound
- CPU bound

- I/O bağımlı ve CPU bağımlı process'ler arasında ayrım yapılması önemlidir.
- En iyi sistem kullanımı için Zamanlayıcı, hem I/O bağımlı hem de CPU bağımlı process'lerin iyi bir karışımını planlamalıdır.



Process Zamanlama (Scheduling)

- Bazı problemlerin çözümü yoğun olarak CPU kullanmayı gerektirir.
- Örneğin bilimsel hesaplama gerektiren bir problemin çözümünü yapan process CPU bağımlı olarak çalışır.
- Web sunucusu (örneğin, Apache web sunucusu) ise ağırlıklı olarak I/O sistemini (hem dosya sistemini hem de bilgisayar ağını) yoğun olarak kullanır. Bu nedenle I/O bağımlı bir process'tir.
- Zamanlayıcı, I/O bağımlı process'leri eğer I/O işlemleri disk üzerinde gerçekleşiyorsa, diski sürekli meşgul tutacak şekilde sıralamalıdır.
- Eğer I/O işlemleri bilgisayar ağı üzerinde gerçekleşiyorsa, zamanlayıcı process'leri bant genişliğinin tamamını kullanacak şekilde sıralamalıdır.

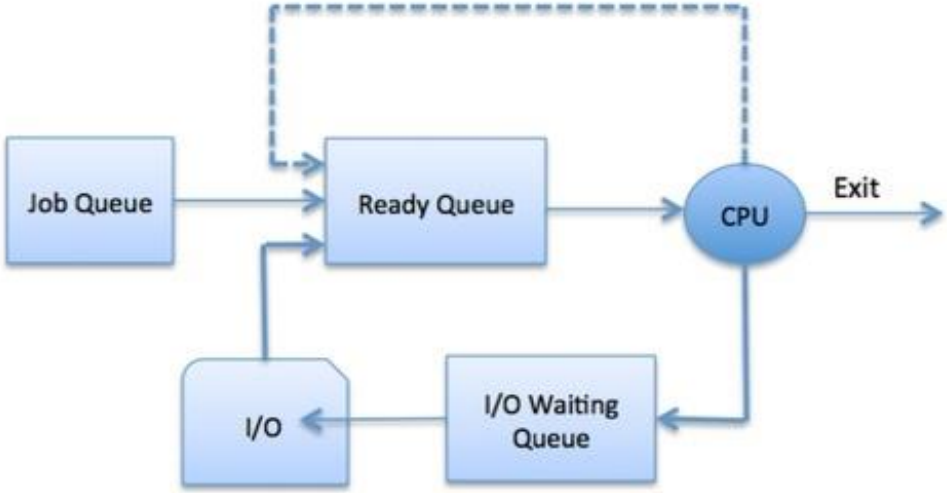
Zamanlayıcı Türleri

- Zamanlayıcıları dört farklı gruba ayırmak mümkündür:

1) Giriş/Çıkış zamanlayıcı (I/O scheduler)
2) Uzun vadeli zamanlayıcı (Long-term scheduler)
3) Orta vadeli zamanlayıcı (Medium-term scheduler)
4) Kısa vadeli zamanlayıcı (Short-term scheduler)

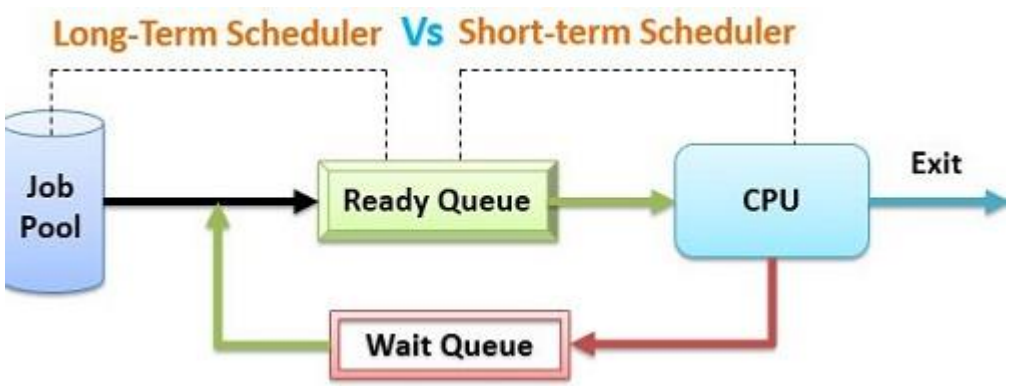
G/Ç Zamanlayıcı (I/O Scheduler)

- Birden fazla process aynı I/O cihazına erişmek isteyebilir.
- Bir I/O cihazının kullanılamaması nedeniyle engellenen process'ler I/O bekleme kuyruğunu (I/O waiting queue) oluşturur.
- Burada hangi process'in I/O cihazına erişeceğine karar verilir.
- I/O bekleyen hangi process'in isteğinin karşılanacağı kararının verilmesidir.



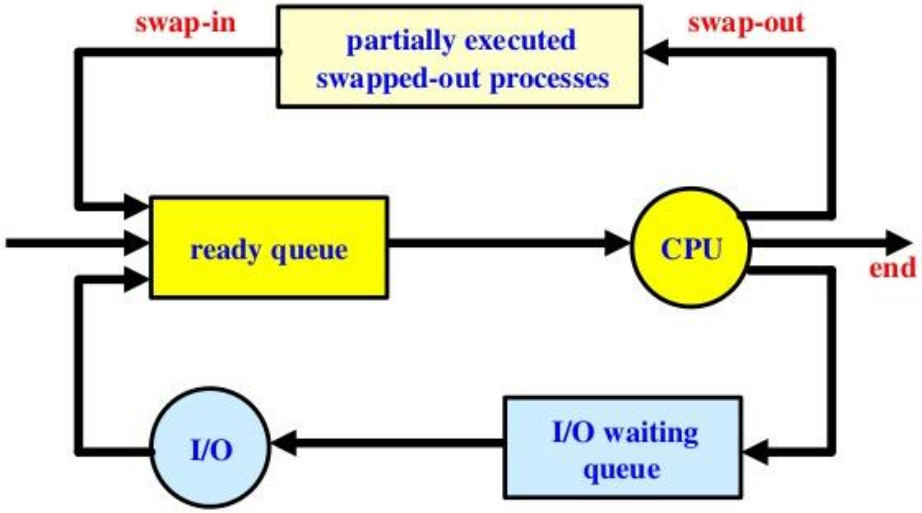
Uzun Vadeli Zamanlayıcı (Long-Term Scheduler)

- Uzun Vadeli Zamanlayıcı, bir iş zamanlayıcısıdır. İş havuzundan hangi process'lerin hazır kuyruğuna (ready queue) alınması gerektiğine karar verir.
- Uzun Vadeli Zamanlayıcı, I/O yoğun ve CPU yoğun process'lerin iyi bir karışımını seçmelidir. Tüm process'ler I/O yoğun ise, hazır kuyruğu çoğu zaman boş olacaktır ve kısa vadeli zamanlayıcı çok az şey yapacaktır. Tüm process'ler CPU yoğun ise, I/O bekleme kuyruğu çoğu zaman boş olacaktır.
- Uzun Vadeli Zamanlayıcı, Kısa Vadeli Zamanlayıcıya kıyasla daha az sıklıkla çalışır.
- Windows ve Unix işletim sistemlerinde uzun vadeli zamanlayıcı (long-term scheduler) bulunmaz ve sadece kısa vadeli zamanlayıcı (short-term scheduler) kullanılır.



Orta Vadeli Zamanlayıcı (Medium-Term Scheduler)

- Orta Vadeli Zamanlayıcı, takas (swap) işleminin bir parçası olarak çalışır. Bu bir process takas zamanlayıcıdır.
- Bir process I/O isteğinde bulunursa, yürütümde olan process askıya (suspend) alınabilir. Askıya alınmış process'ler o an için tamamlanma yolunda ilerleyemez. Bu durumda, process'i bellekten kaldırmak ve diğer process'ler için yer açmak için askıya alınmış process ikincil belleğe taşınır. Bu işlem takas (swap) olarak adlandırılır.
- Orta vadeli zamanlayıcı, takas edilen process'leri idare etmekle görevlidir.



Kısa vadeli zamanlayıcı (Short-term scheduler)

- İşlemciyi kullanmakta olan bir process’in yürütmesi bir şekilde sona erdiğinde, işlemciyi bir sonra kullanacak process’in seçilmesi gerekir.

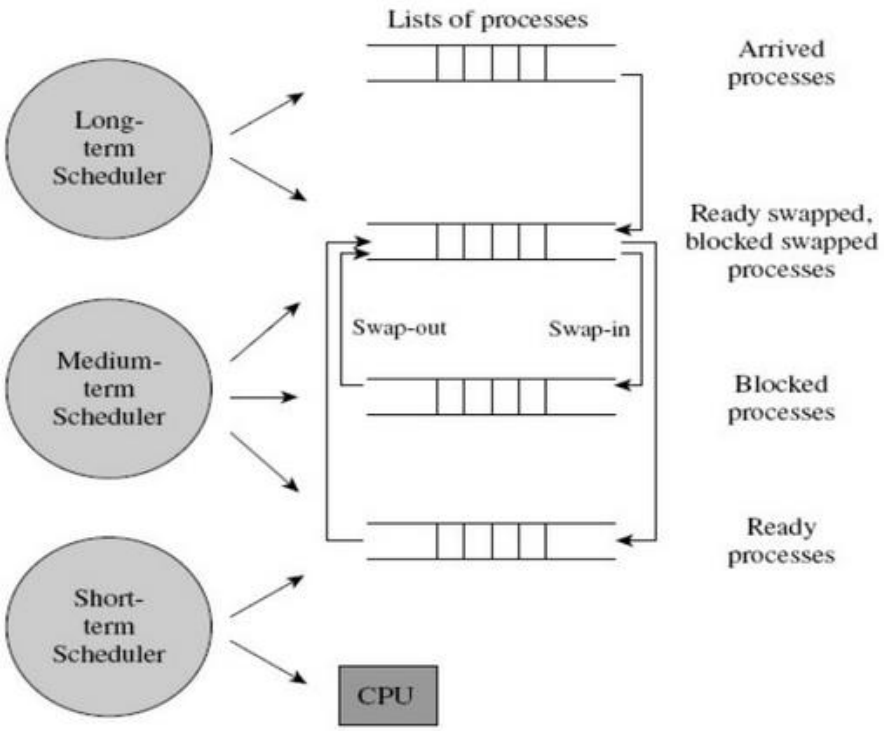
- Kısa vadeli zamanlayıcı, hazır kuyruğunda (ready queue) bulunan ve CPU’yu kullanabilecek durumdaki process’lerden hangisinin seçileceğine karar verir ve CPU'yu seçilen process’e tahsis eder.

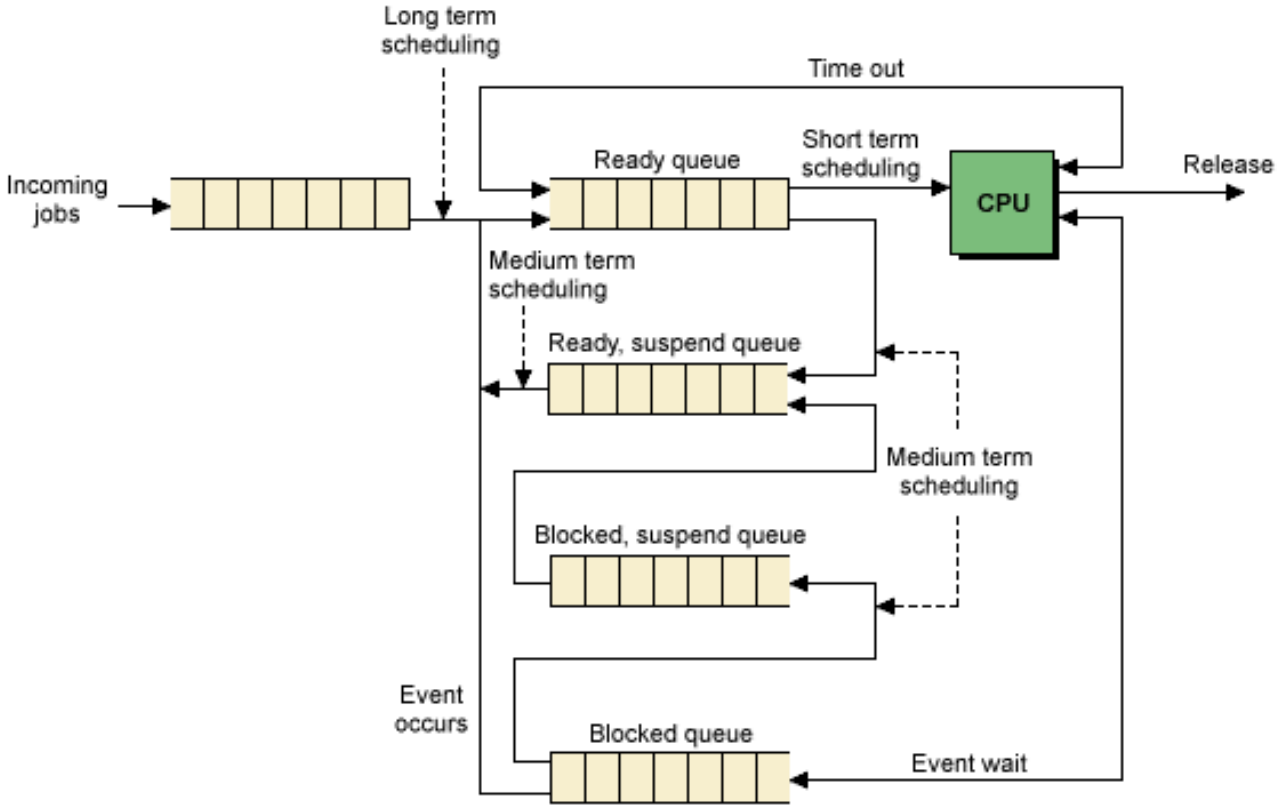
Kısa vadeli zamanlayıcı (Short-term scheduler)

- Kısa vadeli zamanlayıcı, CPU için sık sık yeni bir process seçmelidir.
- Kısa vadeli zamanlayıcının temel amacı, CPU performansını ve process yürütme oranını arttırmaktır.

- Kısa vadeli zamanlayıcı, mevcut process'in yürütülmesinin kesintiye uğramasına yol açabilecek bir olay meydana geldiğinde çağrılır.
- Bu olaylar; Saat-kesmeleri, I/O kesmeleri, sistem çağrıları, sinyaller, gerçek zamanlı sistem politikaları vs. olabilir.

Uzun, orta ve kısa vadeli zamanlama





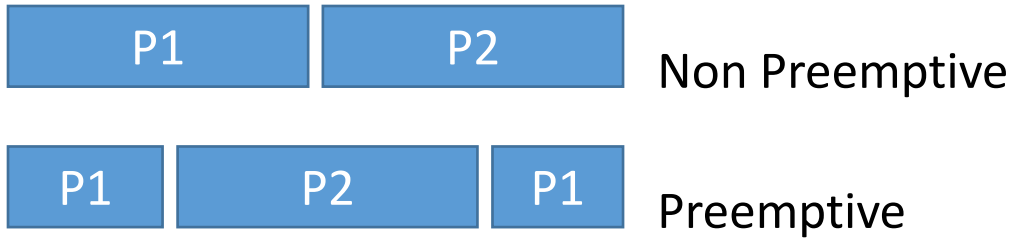
Kısa Vadeli Zamanlama Algoritmalarının Çalışma Türleri

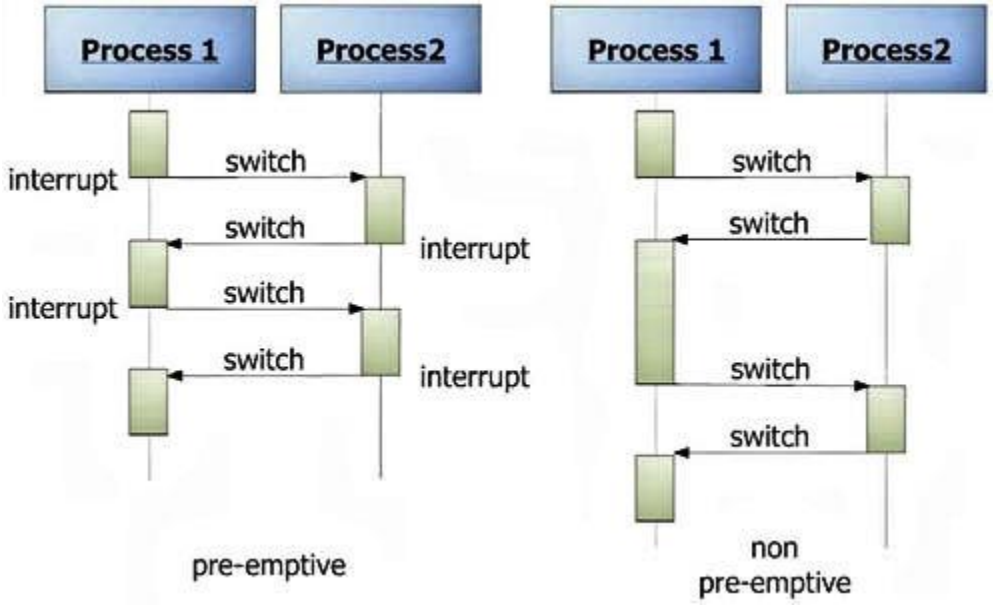
- Hazır kuyruğu (ready queue) ile işlemci arasında zamanlama ilişkisini sağlayan zamanlama algoritmaları (scheduling algorithms) çalışma türlerine göre 2 grupta incelenebilir:

a) Kesintisiz (non-preemptive)

b) Kesintili (preemptive)

Kesintili (Pre-emptive) ve Kesintisiz (Non Preemptive) çalışma





Kesintisiz (Non Preemptive) çalışma :

İşlemci bir process'e tahsis edildiğinde; process kendi kodunda bulunan bir I/O isteği ile bekleme (waiting) durumuna geçtiğinde ya da kendi yürütme süresi tamamlandığında (burst time) ancak işlemciyi serbest bırakır.

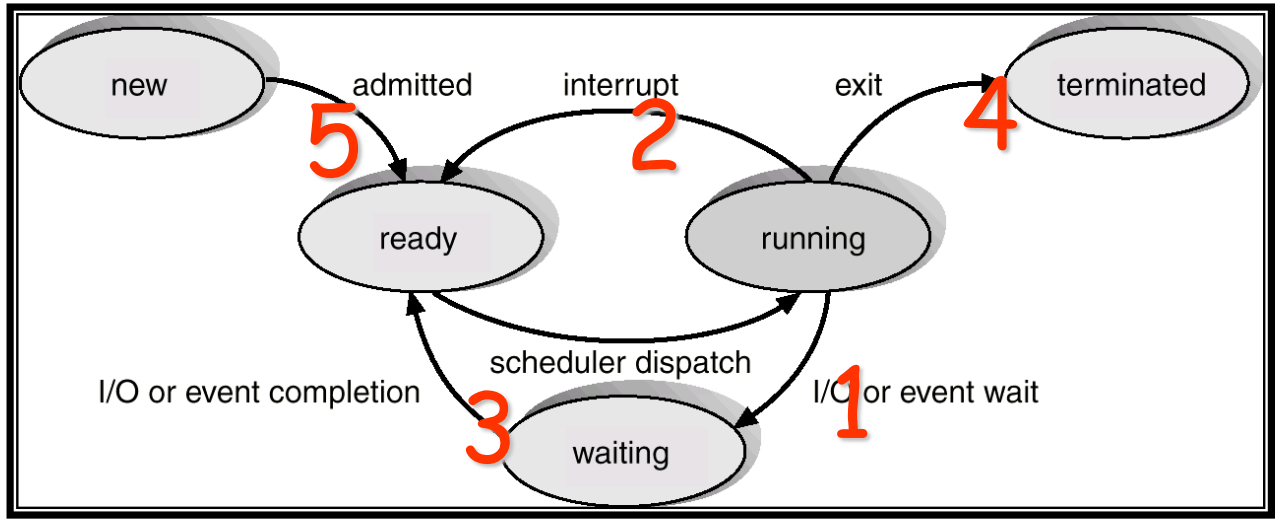
Bu durumların haricinde işlemci başka bir process'e tahsis edilemez.

Kesintili (Preemptive) çalışma :

İşlemci sınırlı bir süre için bir process'e tahsis edilir. Yürütülen process'in işlemciden kaldırılması ve ardından başka bir process'in işlemcide yürütülmesi sağlanabilir.

Kesintisiz (non preemptive) :

- Bu türdeki bir zamanlama algoritmasında bir process, işlemciyi kullanması için seçildikten sonra;
- Process, işletim süresini (burst time) tamamladığında veya kendi kodunda bulunan bir I/O isteği ile bekleme durumuna (waiting state) geçince işlemciden çıkabilir.



- (1) Switches from running to waiting
- (4) Terminated
- Sadece (1) ve (4) koşullarında işlemciyi serbest bırakabilir.
- Bu yöntem, Microsoft Windows 3.x tarafından kullanılmıştır.

Kesintili (preemptive) :

- Bir process'in, kendi isteğinin dışında CPU'yu bırakmak durumunda kalabildiği durumdur.
- Switches from running to ready
- Switches from waiting to ready

Preemption

- Bir process işlemcide yürütülmeye başlandıktan sonra belirli bir süreye kadar çalıştırılır.
- Bu süre bittiğinde, process askıya veya beklemeye alınır ve zamanlayıcı başka bir process'i işlemcide yürütülmesi için seçer.
- Böylece yürütülen process'in CPU'dan kaldırılması ve istenilen başka bir process'in CPU'da yürütülmesi sağlanabilir.

- Bu yöntem, Microsoft Win95 ve Windows işletim sistemlerinin sonraki tüm sürümleri tarafından kullandı.

Farklı İşletim Sistemleri için Zamanlama

- CPU'yu kullanmak için birbiriyle yarışan process'lerin CPU'ya tahsisi, herbir process'e eşit zaman dilimleri verilerek yada process'lere öncelik düzeyi atanarak yapılabilir.

- **Gerçek zamanlı (Real-time)** sistemlerde bir işin belli bir zaman içerisinde gerçekleştirilmesi önemlidir, bu yüzden sürelerin iyi ayarlandığı kesintili (preemptive) zamanlama algoritmaları kullanılmalıdır.

✓Son teslim süresine riayet etme: Veri kaybından sakınmalıdır.

✓Çoklu ortam sistemlerinde nitelik bozulmasından sakınmalıdır.

Farklı İşletim Sistemleri için Zamanlama

- **Zaman paylaşımlı (Time-sharing)** sistemlerde CPU'nun process'lere veya kullanıcılara eşit olarak paylaştırılması,

✓Her process işlemciyi adil kullanmalıdır.

- **İnteraktif sistemler (Interactive systems)** kullanıcının sürekli veri alıp verdiği sistemlerdir. Bu yüzden kullanıcının verdiği komutlara alınan cevap süresi önemlidir. Kesintili (preemptive) zamanlama zorunludur.

✓Cevap Süresi: İsteklere hızlı şekilde cevap verilmelidir.

✓Orantılı Olma: Kullanıcının beklentilerini karşılamalıdır.

Farklı İşletim Sistemleri için Zamanlama

- **Toplu işlemler (Batch)** ise bir işin tamamlanma süresi (çünkü kullanıcı ancak bu süre sonunda cevap alır) ön plandadır.
- Kullanıcılar acil olarak cevap beklemedikleri için uzun süreli kesintili (preemptive) ya da kesintisiz (non preemptive) zamanlama algoritmaları kullanılabilir. Process değişimi az olduğu için performans artar.

✓Üretilen İş: Saat başına üretilen iş maksimum olmalıdır.

✓Dönüş Süresi: Sürecin sunumu ve sonlanması arasındaki sürenin minumum olmasıdır.

✓İşlemci Kullanımı: İşlemci her zaman meşgul olmalıdır.

Process Zamanlama Kriterleri için bazı tanımlar

- **Varış Zamanı (Arrival Time) :** Process'in hazır kuyruğuna geldiği zaman.
- **İşletim süresi (Burst Time):** CPU process yürütme işleminin gerektirdiği süre.
- **Tamamlama Süresi (Completion Time) :** Process'in yürütmesini tamamladığı zaman.
- **Yapılan İş hacmi (Throughput) :** Birim zaman diliminde yürütmesi tamamlanan process sayısı.

Process Zamanlama Kriterleri için bazı tanımlar

- **Dönüş Süresi (Turn-around time):** Bir process'in oluşturulmasından çalışmasının sonlanmasına kadar geçen süre,

- **Bekleme Süresi (Waiting time):** Process'in hazır kuyruğundaki (ready queue) geçirdiği toplam süredir, (I/Oiçin aygıt beklemek sayılmaz)

- **Cevap Süresi (Response time):** Process'in, hazır kuyruğuna geldikten sonra, işlemciyi ilk olarak kullanmaya başladığı zamana kadar geçen süre.

Process Zamanlama Performans Kriterleri

- Maximize CPU Utilization
- Maximize Throughput
- Minimize Turn-around time
- Minimize Waiting time
- Minimize Response time

Process Zamanlama Performans Kriterleri

- ***Maksimum CPU kullanımı (Max CPU utilizitation):***

- CPU'yu olabildiğince meşgul tutun
- CPU yararı 0’dan 100’e kadar değişebilmektedir
- (Normal sistem: 40, yoğun sistem: 90)

Process Zamanlama Performans Kriterleri

- ***Maksimum İş Hacmi (Max Throughput):***

- Birim zamanda yürütme işlemini tamamlayan process sayısı.

- Birim zamanda tamamlanan toplam process sayısı sistem başarısını gösteren bir kriterdir.

Process Zamanlama Performans Kriterleri

- ***Minimum Dönüş Süresi (Min Turn-around time):***
- Bir process'in oluşturulmasından çalışmasının sonlanmasına kadar geçen süre.
- Process için yürütülme ve bekleme sürelerinin toplamıdır.
- Process belleğe yüklenirken, kuyrukta beklerken, CPU'da yürütülürken ve I/O işlemleri tamamlanırken geçen sürelerin tamamıdır.

Dönüş süresi = Tamamlanma zamanı – Kuyruğa varış zamanı

veya

Dönüş süresi = İşletim süresi + Bekleme süresi

TurnaroundTime = CompletionTime- ArrivalTime

TurnaroundTime = BurstTime + WaitingTime

Process Zamanlama Performans Kriterleri

- ***Minimum Bekleme Süresi (Min Waiting time):***
- Process'in, hazır kuyruğunda (ready queue) geçirdiği toplam süre bekleme süresi olarak adlandırılır.

Bekleme süresi = Dönüş süresi – İşletim süresi

Waiting Time = Turn Around Time – Burst Time

Process Zamanlama Performans Kriterleri

- ***Minimum Cevap Süresi (Min Response time):***
- *Cevap Süresi,* Process'lerin, CPU kullanımı için hazır (ready) kuyruğuna geldikten sonra, işlemciyi ilk olarak kullanmaya başladığı zamana kadar geçen süredir.
- ***Cevap Süresi* = İşlemciyi ilk kullanma zamanı – Kuyruğa varış zamanı**
- ***Response time* = Time at which the process gets the CPU for the first time - Arrival time**
- Örneğin;
- P1 process'i hazır kuyruğuna 0. zamanda ulaşmış ve 0. zamanda işlemcide ilk kez yürütülmeye başlamıştır. Bu durumda cevap süresi (response time) 0 birimdir.
- P2 process'i hazır kuyruğuna 0. zamanda ulaşmış ve 2. zamanda işlemcide ilk kez yürütülmeye başlamıştır. Bu durumda cevap süresi (response time) 2 zaman birimidir.

Kısa vadeli Zamanlama Algoritmaları (Scheduling Algorithms)

- **İlk Gelen Önce** (First Come First Served-FCFS)
- **Zaman Dilimli** (Round Robin-RR)
- **En Kısa İş Önce** (Shortest Job First-SJF)
- **En Kısa İşletim Süresi Kalan** (Shortest Remaining Time-SRT)
- **Öncelik Tabanlı** (Priority)
- **En Yüksek Cevap Oranı Sonraki** (Highest Response Ratio Next-HRRN)
- **Çok Düzeyli Kuyruk** (Multilevel Queue Scheduling)
- **Çok Düzeyli Geri Bildirim Kuyruğu** (Multilevel Feedback Queue Scheduling)

İlk Gelen Önce (First Come First Served - FCFS)

- Diğer bir adı ile de FIFO (First in- First out).
- En basit zamanlama algoritmasıdır.
- **Kesintisiz (non-preemptive)** türdeki bir zamanlama algoritmasıdır.
- Bu sistemde hazır kuyruğuna ilk gelen process CPU'ya yerleştirilir.
- Process'ler hazır kuyruğunda (ready queue) geliş sıralarına göre dizilirler. Yeni gelen process sıra en sonuna yerleştirilir.
- Process'ler hazır kuyruğuna (ready queue) geliş sırasına göre işlemcide yürütülür.

İlk Gelen Önce (First Come First Served - FCFS)

- Yürütülen process bloklanınca bekleme kuyruğuna gönderilir. Daha sonra hazır kuyruğundaki sıranın en başındaki process işlemciye atanır.
- Bloklanmış process tekrar hazır durumuna geçtiğinde yeni bir process gibi davranılarak hazır kuyruğunun sonuna eklenir.

- Yürütülecek bir process'in seçilmesi, yalnızca hazır kuyruğunun ön kısmından bir process'in kaldırılmasını gerektirir.

İlk Gelen Önce (First Come First Served - FCFS)

- Bu algoritmanın en büyük avantajı anlaşılmasının ve programlanmasının kolay olmasıdır.
- Fakat bu zamanlama algoritmasının, kısa süreli ve çoğunlukla veri alışverişi yapan process'ler için performansı çok kötüdür.
- Uzun processler yüzünden kısa process'ler gereksiz beklerler.
- Etkileşimli process'ler varsa etkin bir yöntem değildir, uzun cevap süreleri oluşabilir.
- I/O yoğun process'ler için uygun değildir.
- İlk Gelen Önce algoritması genelde diğer yöntemlerle birlikte kullanılır.
- Bu algoritma aynı öncelikli process'lerin kendi aralarında sıralaması için kullanılabilir.

İlk Gelen Önce (First Come First Served - FCFS)

- **Örnek:**
- t=0'da hazır kuyruğuna P1, P2, P3, P4 ve P5 sırasında gelen 5 process olsun ve bu process'lerin işletim süreleri (burst time) T ile gösterilsin. Her bir process'in dönüş süresini (turn-around time) ve bekleme süresini (waiting time) İlk Gelen Önce (First Come First Served - FCFS) algoritmasına göre hesaplayınız.

Process	P1	P2	P3	P4	P5
İşletim Süresi (Burst Time) T(Pi)	200	110	300	150	50

Çözüm

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime

- P1: 200 – 0 = 200
- P2: 310 – 0 = 310
- P3: 610 – 0 = 610
- P4: 760 – 0 = 760
- P5: 810 – 0 = 810
- Ortalama dönüş süresi =Σ(dönüş(Pi))/5 = (200+310+610+760+810)/5=538

Process	P1	P2	P3	P4	P5
işletim Süresi (burst time) T(Pi)	200	110	300	150	50

Gantt Şeması

200	310	610	760	810
P1	P2	P3	P4	P5

Çözüm

- **Herbir process'in bekleme süresi (waiting time)**;
- Waiting Time = Turn Around Time – Burst Time

- P1: 200 – 200 = 0
- P2: 310 – 110 = 200
- P3: 610 – 300 = 310
- P4: 760 – 150 = 610
- P5: 810 – 50 = 760
- Ortalama bekleme süresi =Σ(bekleme(Pi))/5 = (0+200+310+610+760)/5=376

Process	P1	P2	P3	P4	P5
işletim Süresi (burst time) T(Pi)	200	110	300	150	50

Gantt Şeması

200	310	610	760	810
P1	P2	P3	P4	P5

İlk Gelen Önce (First Come First Served FCFS)

- **Örnek:**
- Process'lerin, aşağıdaki tabloda verilen varış sürelerine (arrival time) göre hazır kuyruğuna (ready queue) katılacakları kabul edilirse, İlk Gelen Önce (First Come First Served-FCFS) algoritmasına göre process'lerin dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.

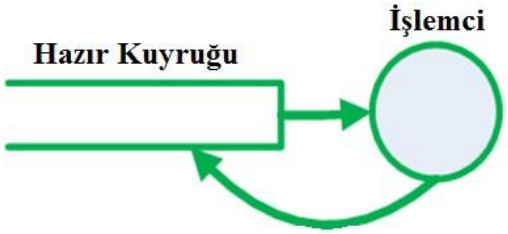
Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5

Çözüm

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime
- P1: 16 – 0 = 16
- P2: 19 – 1 = 18
- P3: 24 – 2 = 22

- Ortalama dönüş süresi (toplam dönüş süresi/process sayısı):
- Ortalama dönüş süresi = (16 + 18 + 22)/3 = 18.66

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	**0**	**16**
P2	**1**	**3**
P3	**2**	**5**



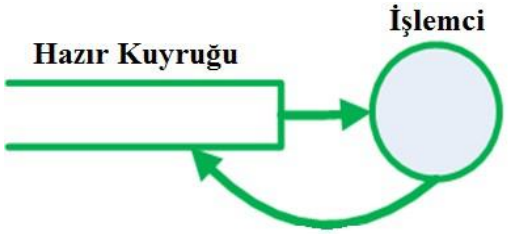
Gantt Şeması

Çözüm

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time
- P1: 16 – 16 = 0
- P2: 18 – 3 = 15
- P3: 22 – 5 = 17

- Ortalama bekleme süresi (toplam bekleme süresi/process sayısı)
- Ortalama bekleme süresi = (0 + 15 + 17)/3 = 10.7ms

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	**0**	**16**
P2	**1**	**3**
P3	**2**	**5**



Gantt Şeması

Zaman Dilimli (Round Robin)

- **Kesintili (preemptive)** türdeki bir zamanlama algoritmasıdır.

- Zaman Dilimli (Round Robin) algoritmasında process'ler eşit uzunluktaki zaman dilimlerinde işlemciye (CPU) sırayla anahtarlanırlar.

- İlk Gelen Önce algoritmasında olduğu gibi process'ler hazır kuyruğunda dizilir.

- Yürütülecek her process için **quantum süresi (q)** denilen sabit bir process süresi ayrılır ve CPU'ya gelen her process en çok bu süre kadar yürütülür.

Zaman Dilimli (Round Robin)

- Bir process’den başka bir process’e geçiş işlemide (context switching) sistemde belirli bir süre almaktadır.

- Örneğin; Process’ler arası değişim işleminin 1ms aldığını varsayalım. Quantum süresi 4ms seçilirse, context switching 1ms alır. Dolaysıyla İşlemcinin %20 si yönetimsel ek yük işlemleri için harcanır.

- İşlemci etkinliğini arttırmak için, quantum süresi 100ms seçilirse, yönetimsel ek yük %1 olacaktır.

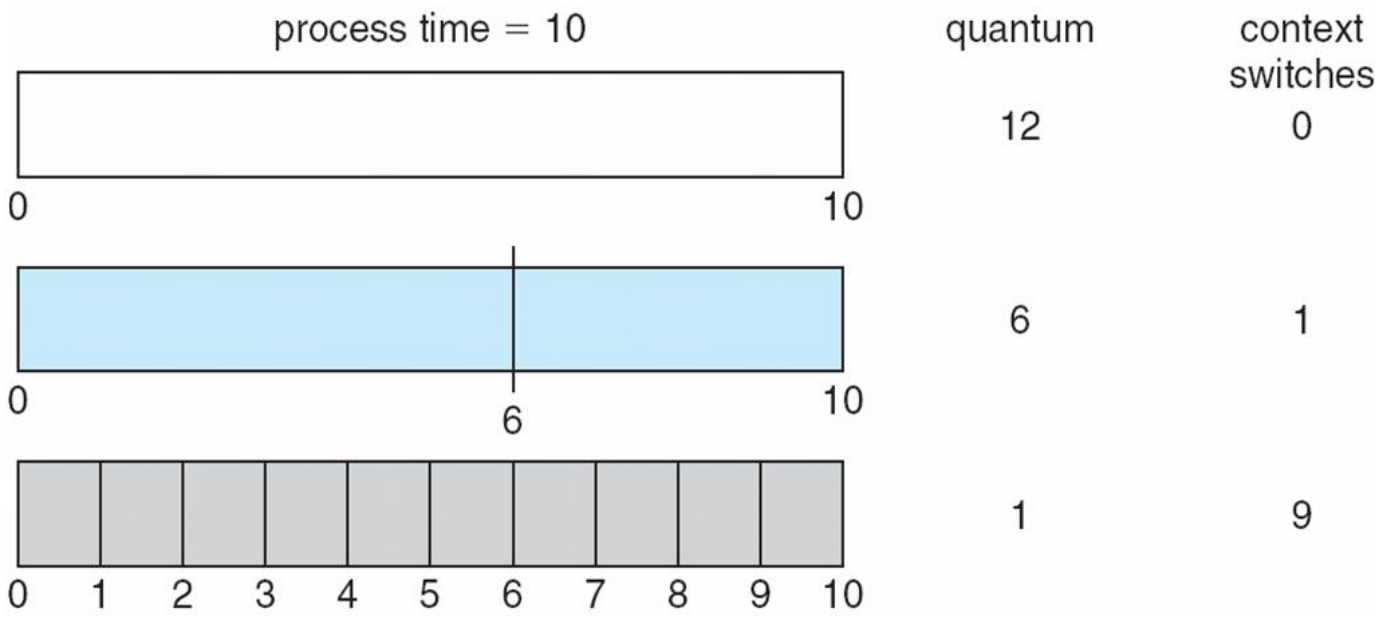
Zaman Dilimli (Round Robin)

- Zaman Dilimli (Round Robin) algoritmada en önemli konu quantum süresidir.

- Quantum süresini çok kısa tutmak context switching işlemi için çok zaman kaybı olacağından işlemci etkinliğini azaltır.
- *Quantum time small $\Rightarrow$ işlemci etkinliğini azaltır, quantum time context switching süresine göre büyük olmalıdır.*

- Quantum süresini çok uzun tutmak ise isteklere verilen cevabı azaltır ve İlk Gelen Önce (First Come First Served - FCFS) algoritmasından bir farkı kalmaz process sonlanır boş zaman kalması bile söz konusu olur.
  - *q very large $\Rightarrow$ FIFO*

Quantum Time and Context Switching



küçük quantum zamanı context switching işlemini arttırır.

Zaman Dilimli (Round Robin)

- Eğer process, quantum süresinin sonunda hala bitmezse CPU’daki yürütümü durdurularak tekrar hazır kuyruğunun sonuna eklenir ve başka bir process’e geçilir.

- Process’in quantum süresi bitmeden, bir I/O olayı için process bekleme (waiting) durumuna geçerse ya da çalışması sonlanırsa işlemciye başka bir process atanır.

Zaman Dilimli (Round Robin-RR)

- **Örnek:**
- Process'lerin, aşağıdaki tabloda verilen varış sürelerine (arrival time) göre hazır kuyruğuna (ready queue) katılacakları kabul edilirse, quantum süresi 4ms (q=4) olan bir Zaman Dilimli (Round Robin) zamanlama algoritmasına göre process'lerin dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.

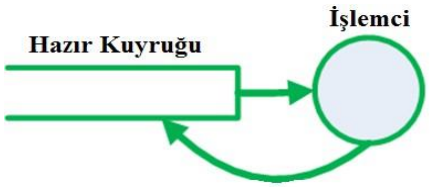
Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5

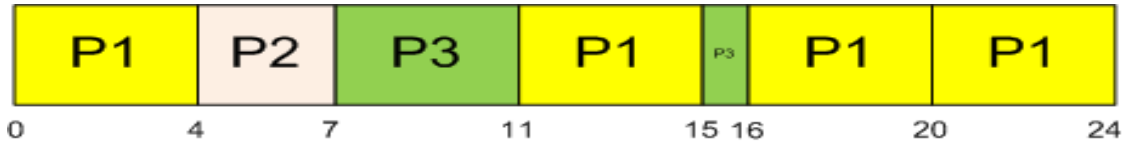
Zaman Dilimli (Round Robin-RR)

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime
- P1: 24 – 0 = 24
- P2: 7 – 1 = 6
- P3: 16 – 2 = 14

- Ortalama dönüş süresi (toplam dönüş süresi/process sayısı):
- Ortalama dönüş süresi = (24 + 6 + 14)/3 = 14.6

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5





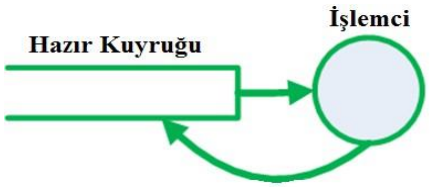
Gantt Şeması

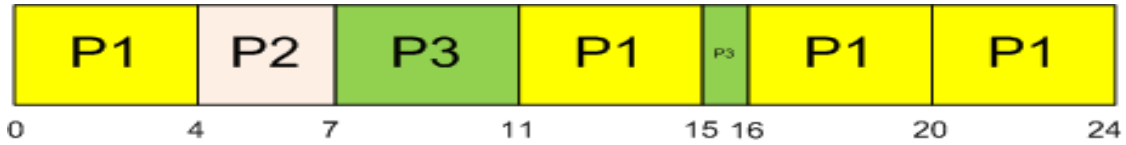
Zaman Dilimli (Round Robin-RR)

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time
- P1: 24 – 16= 8
- P2: 6 – 3 =3
- P3: 14 – 5 =9

- Ortalama bekleme süresi (toplam bekleme süresi/process sayısı):
- Ortalama bekleme süresi = (8 + 3 + 9)/3 = 6.66

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5





Gantt Şeması

En Kısa İş İlk Önce (Shortest Job First-SJF)

- **Kesintisiz (non pre-emptive)** türdeki bir zamanlama algoritmasıdır.

- Her bir process, bir sonraki CPU kullanım süresinin uzunluğu ile ilişkilendirilir. Bu uzunluklar kullanılarak sonlanmak için en kısa süreye ihtiyaç duyan process ilk hizmet görür.

- Process'lerin toplam işletim sürelerinin belirli olduğu düşünülür. Zamanlayıcı, hazır kuyruğunda bulunan process'lerden en kısa sürede tamamlanacak olan process'i seçer.

En Kısa İş İlk Önce (Shortest Job First-SJF)

- Ortalama bekleme süresi düşünüldüğünde SJF optimumdur. Verilmiş bir process serisi için en düşük bekleme sürelerini elde edebilir.

- Kısa işlere iyi hizmet verir. Etkileşimli ortamlara uygun değildir.

- Bu yöntemin problemi, process'in sonlanması için ne kadar süre CPU kullanması gerektiğinin tahmin edilmesi gerekir.

En Kısa İş İlk Önce (Shortest Job First-SJF)

- **Örnek:**
- t=0'da hazır kuyruğuna (ready queue) P1, P2, P3, P4 ve P5 sırasında gelen 5 process olsun ve bu process'lerin işletim süreleri T ile gösterilsin. Her bir process'in dönüş süresini (turn around time) ve bekleme süresini (waiting time) En Kısa İş (Shortest Job First) algoritmasına göre hesaplayınız.

Process	P1	P2	P3	P4	P5
işletim süresi (burst time) T(Pi)	200	110	300	150	50

Çözüm :

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime

- P1: 510 – 0 = 510
- P2: 160 – 0 = 160
- P3: 810 – 0 = 810
- P4: 310 – 0 = 310
- P5: 50 – 0 = 50
- Ortalama dönüş süresi =Σ(dönüş(Pi))/5 = (510+160+810+310+50)/5=368

Process	P1	P2	P3	P4	P5
işletim süresi (burst time) T(Pi)	200	110	300	150	50

Gantt Şeması

0	50	160	310	510	810
P5	P2	P4	P1	P3	

Çözüm :

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time

- Tbekleme(P1)= 510 – 200 = 310
- Tbekleme(P2)= 160 – 110 = 50
- Tbekleme(P3)= 810 – 300 = 510
- Tbekleme(P4)= 310 – 150 = 160
- Tbekleme(P5)= 50 – 50 = 0
- Ortalama bekleme süresi = Σ(Tbekleme(Pi))/5 = (310+50+510+160+0)/5=206

Process	P1	P2	P3	P4	P5
işletim süresi (burst time) T(Pi)	200	110	300	150	50

Gantt Şeması

0	50	160	310	510	810
P5	P2	P4	P1	P3	

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- En Kısa İş ilk önce (Shortest Job First) algoritmasının **kesintili (preemptive)** bir uyarlamasıdır.
- Kuyruğa yeni gelen process'lerde göz önüne alınarak, sonlanmasına en az süre kalan process önce yürütülür.
- Kuyruğa yeni bir process geldiğinde, mevcutta işlemcide yürütülen process'in kalan süresi ile yeni process'in kalan süresi karşılaştırılır ve küçük olan tercih edilir.
- Mevcutta yürütülen bir process yeni gelen kısa bir process yüzünden kesintiye uğrayabilir.

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- En kısa işletim süresi kalan process'e öncelik sağlanarak biran önce sistemden çıkması ve bu yolla en kısa ortalama bekleme süresinin elde edilmesi amaçlanır.
- Kısa process'ler çabucak hizmet görebilir fakat uzun processler çok bekleyebilir.

- Zaman kaybı fazladır, çünkü kullanılan ve kalan süre hesabı yapılır ve kesintide context switching vakit kaybettirir.

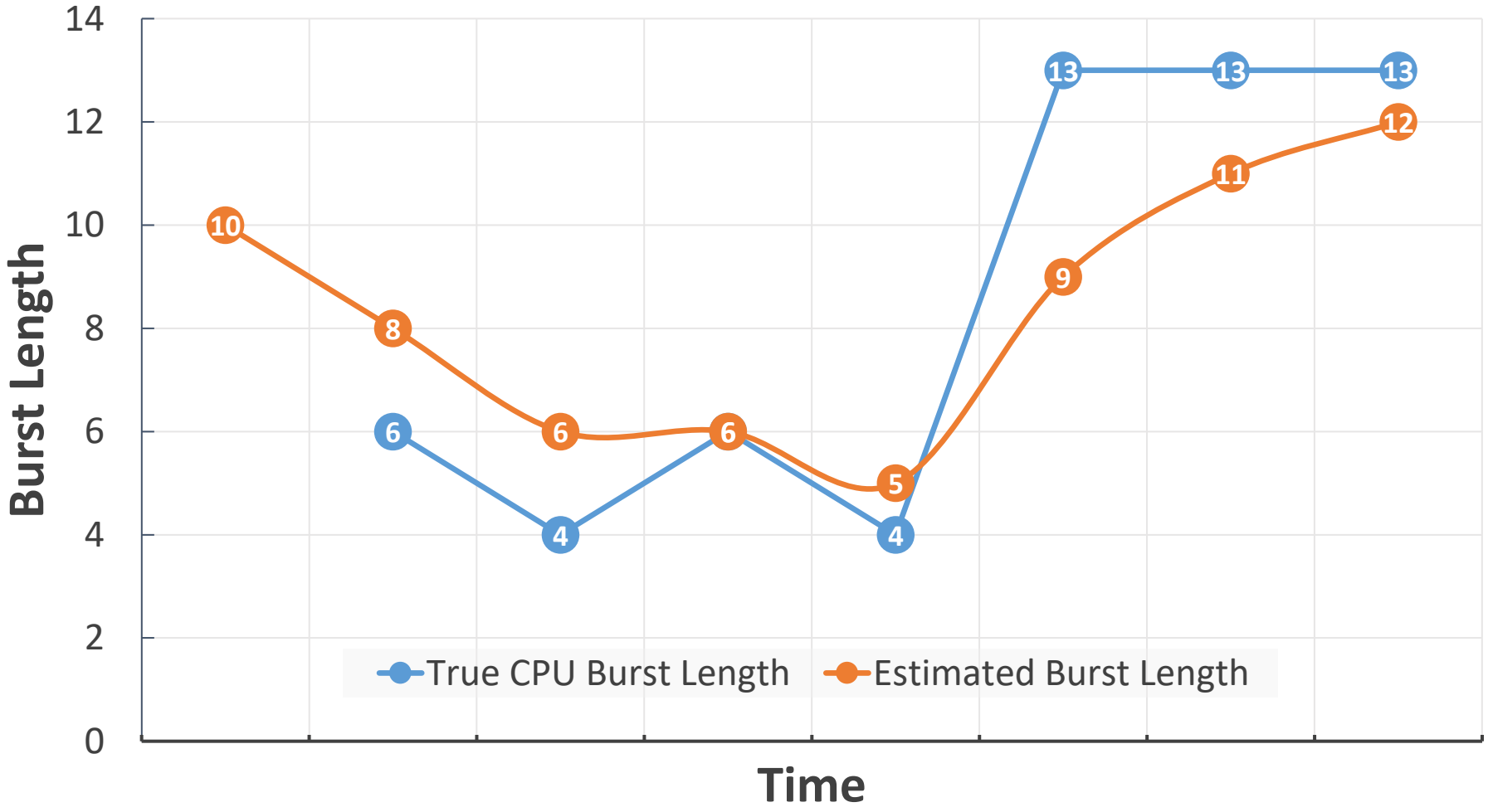
- Process'lerin sonlanma süreleri bilinmelidir.
- Kalan işletim süresi tam olarak ölçülebilen bir değer değildir!

Bir Sonraki CPU İşletim Süresini Belirleme

- Sorun: Gelecekteki CPU işletim süreleri bilinmiyor olabilir.
- Çözüm: Bir önceki işletim süresine göre bir sonraki işletim süresini tahmin etmek.
- Process davranışının çok değişken olmadığı varsayılır.
- Üstel ortalaması (exponential averaging) kullanılır.

- t_n – Ölçülen n. CPU işletim süresi
- τ_{n+1} – tahmin edilen n+1. CPU işletim süresi
- α – ağırlık faktörü $(0 \leq \alpha \leq 1)$
- $\boldsymbol{\tau_{n+1} = \alpha t_n + (1 - \alpha)\, \tau_n}$
- estimate[n + 1] = alpha * burst time[n] + (1 - alpha) * estimate[n]
- Tipik olarak, $\alpha = 0{,}5$

Gerçek and Tahmin edilen CPU İşletim Süreleri



Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Örnek:**
- Process'lerin, aşağıdaki tabloda verilen varış sürelerine göre hazır kuyruğuna katılacakları kabul edilirse, **Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)** zamanlama algoritmasına göre process'lerin dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.

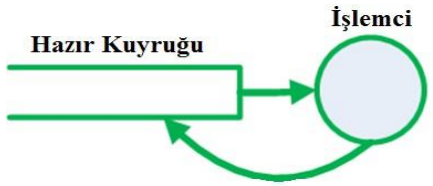
Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	0	3
P3	0	5

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime
- P1: 24 – 0 = 24
- P2: 3 – 0 = 3
- P3: 8 – 0 = 8

- Ortalama dönüş süresi (toplam dönüş süresi/process sayısı):
- Ortalama dönüş süresi = (24 + 3 + 8)/3 = 11.6

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	0	3
P3	0	5



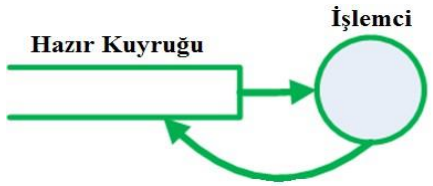
Gantt Şeması

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time
- P1: 24 – 16 = 8
- P2: 3 – 3 = 0
- P3: 8 – 5 = 3

- Ortalama bekleme süresi (toplam bekleme süresi/process sayısı):
- Ortalama bekleme süresi = (8 + 0 + 3)/3 = 3.7ms

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	0	3
P3	0	5



Gantt Şeması

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Örnek:**
- Process'lerin, aşağıdaki tabloda verilen varış sürelerine göre hazır kuyruğuna katılacakları kabul edilirse, **Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)** zamanlama algoritmasına göre process'lerin dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.

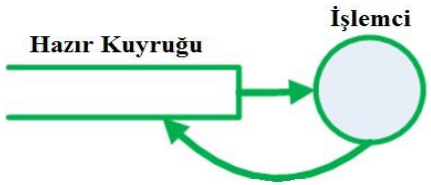
Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime
- P1: 24 – 0 = 24
- P2: 4 – 1 = 3
- P3: 9 – 2 = 7

- Ortalama dönüş süresi (toplam dönüş süresi/process sayısı):
- Ortalama dönüş süresi = (24 + 3 + 7)/3 = 11.33

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5



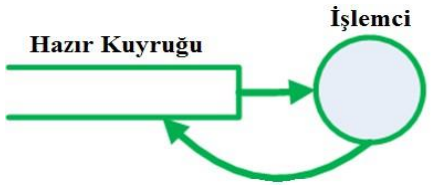
Gantt Şeması

Kalan süresi en kısa olan ilk önce (Shortest-Remaining-Time-First –SRTF)

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time
- P1: 24 – 16 = 8
- P2: 3 – 3 = 0
- P3: 7 – 5 = 2

- Ortalama bekleme süresi (toplam bekleme süresi/process sayısı):
- Ortalama bekleme süresi = (8 + 0 + 2)/3 = 3.3

Process	Varış süresi (arrival time)	işletim süresi (burst time)
P1	0	16
P2	1	3
P3	2	5



Gantt Şeması

Öncelik Zamanlama (Priority Scheduling)

- Her process'e bir öncelik (priority) değeri atanır.
- Yeni bir process işlemcide yürütülmesi için seçileceği zaman, en yüksek önceliğe sahip process'in yürütülmesine izin verilir.
- Eşit önceliğe sahip process'ler ise FCFS sırasıyla atanır.
- Öncelik değeri Process Control Block içinde öncelik (priority) alt alanında tutulur.
- **Düşük öncelik rakamı daha büyük önceliği ifade eder.**
- SJF algoritması aslında bir nevi öncelik zamanlama algoritmasıdır. Öncelik değeri, process işlem süresinin tersi ile elde edilmektedir.

Öncelik Zamanlama (Priority Scheduling)

- **Kesintisiz (non preemptive) ve kesintili (preemptive)** olarak uygulanabilir.
- Non preemptive yönteminde, bir process hazır kuyruğuna geldiğinde, çalışmakta olan process'ten daha öncelikli bile olsa, çalışmakta olan process durum değiştirene kadar yürütülmeye devam eder.

- Preemptive yönteminde, bir process hazır kuyruğuna geldiğinde, işlemcide yürütülmekte olan process'ten daha öncelikli ise, mevcut yürütülmekte olan process kesilir.

Öncelik Zamanlama (Priority Scheduling)

- Öncelik Zamanlama (Priority Scheduling) algoritmasında, CPU sürekli yüksek öncelikli process'leri yürütebilir ve bazı processler sürekli hazır kuyruğunda bekleyebilir (indefinite blocking, starvation).

- Sınırsız beklemeyi engellemek için düşük öncelikli process'ler kuyrukta beklerken öncelik seviyesi artırılır (Örn. her 15 dakikada 1 artırılır).

- Öncelik değeri artırılarak en düşük önceliğe sahip process'in bile belirli bir süre sonunda işlemcide yürütülmesi sağlanır.

Öncelik Zamanlama (Priority Scheduling)
Kesintisiz (non pre-emptive)

- **Örnek:**
- 5 process'e ait işletim süreleri ve öncelik düzeyleri aşağıdaki gibi olsun. Kesintisiz Öncelik Zamanlama (Priority Scheduling) algoritmasına göre Gantt şemasını oluşturunuz. Ayrıca herbir process'in dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.
- (Tüm process'lerin t=0'da hazır kuyruğuna geldiği farzedilecektir).

Process	Varış süresi (arrival time)	işletim süresi (burst time)	Öncelik
P1	0	200	4
P2	0	110	1
P3	0	300	3
P4	0	150	5
P5	0	50	2

Çözüm :

- P2 process'i en yüksek öncelikli olduğundan ilk önce işletilecektir. Daha sonra da sırasıyla P5, P3, P1 ve P4 işletilecektir.
- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime

Process	Varış süresi (arrival time)	işletim süresi (burst time)	Öncelik
P1	0	200	4
P2	0	110	1
P3	0	300	3
P4	0	150	5
P5	0	50	2

- P1 : 660 – 0 = 660
- P2 : 110 – 0 = 110
- P3 : 460 – 0 = 460
- P4 : 810 – 0 = 810
- P5 : 160 – 0 = 160
- Ortalama dönüş süresi = Σ(dönüş(Pi))/5 = (660+110+460+810+160)/5= 440

	0	110	160	460	660	810
Gantt Şeması	P2	P5	P3	P1	P4	

Çözüm :

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time

- P1: 660 – 200 = 460
- P2: 110 – 110 = 0
- P3: 460 – 300 = 160
- P4: 810 – 150 = 660
- P5: 160 – 50 = 110
- Ortalama bekleme süresi = Σ(Tbekleme(Pi))/5 = (460+0+160+660+110)/5=278

Gantt Şeması

0	110	160	460	660	810
P2	P5	P3	P1	P4	

Öncelik Zamanlama (Priority Scheduling)
Kesintisiz (non pre-emptive)

- **Örnek:**
- Process'lerin, aşağıdaki tabloda verilen varış sürelerine göre hazır kuyruğuna katılacakları kabul edilirse, kesintisiz öncelik zamanlama algoritmasına (priority scheduling) göre Gantt şemasını oluşturunuz. Ayrıca herbir process için dönüş süresini (turn around time) ve bekleme süresini (waiting time) hesaplayınız.

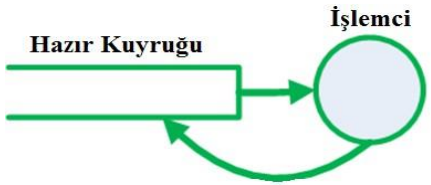
Process	Varış süresi (arrival time)	işletim süresi (burst time)	Öncelik
P1	0	7	3
P2	0	10	1
P3	0	6	4
P4	0	15	2
P5	0	6	0

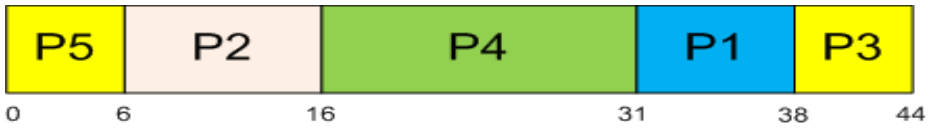
Çözüm

- **Her bir process'in dönüş süresi (turn around time)**;
- TurnaroundTime = CompletionTime - ArrivalTime
- P1: 38 – 0 = 38
- P2: 16 – 0 = 16
- P3: 44 – 0 = 44
- P4: 31 – 0 = 31
- P5: 6 – 0 = 0

- Ortalama dönüş süresi (toplam dönüş süresi/process sayısı):
- Ortalama dönüş süresi = (38 + 16 + 44 + 31 + 0)/5 = 25.8

Process	Varış süresi (arrival time)	işletim süresi (burst time)	Öncelik
P1	0	7	3
P2	0	10	1
P3	0	6	4
P4	0	15	2
P5	0	6	0





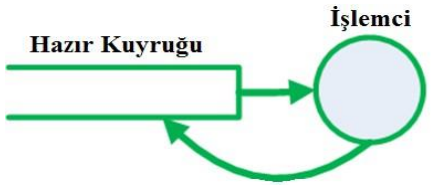
Gantt Şeması

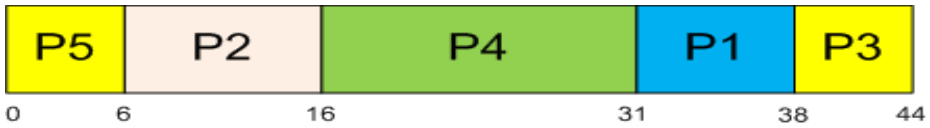
Çözüm

- **Herbir process'in bekleme süresi (waiting time);**
- Waiting Time = Turn Around Time – Burst Time
- P1: 38 – 7 = 31
- P2: 16 – 10 = 6
- P3: 44 – 6 = 38
- P4: 31 – 15 = 16
- P5: 6 – 6 = 0

- Ortalama bekleme süresi (toplam bekleme süresi/process sayısı):
- Ortalama bekleme süresi = (31 + 6 + 38 + 16 + 0)/5 = 18.2

Process	Varış süresi (arrival time)	işletim süresi (burst time)	Öncelik
P1	0	7	3
P2	0	10	1
P3	0	6	4
P4	0	15	2
P5	0	6	0





Gantt Şeması

Öncelik Zamanlama (Priority Scheduling)

Örnek:

Process'lerin, aşağıdaki tabloda verilen varış sürelerine göre hazır kuyruğuna katılacakları kabul edilirse, Kesintisiz (non pre-emptive) ve kesintili (preemptive) öncelik zamanlama (priority) algoritmasına göre process'lerin Gantt grafiklerini gerçekleştiriniz.

Process ID	Kuyruğa varış Zamanı (Arrival Time)	İşletim süresi (Burst time)	Öncelik (Priority)
P1	2	7	9
P2	3	5	7
P3	4	2	3
P4	6	1	10

Çözüm

Process ID	Kuyruğa varış Zamanı (Arrival Time)	İşletim süresi (Burst time)	Öncelik (Priority)
P1	2	7	9
P2	3	5	7
P3	4	2	3
P4	6	1	10

Priority (Preemptive)

	P1	P2	P3	P2	P1	P4

2 3 4 6 10 16 17

Priority (Non Preemptive)

	P1	P3	P2	P4

2 9 11 16 17

Highest Response Ratio Next (HRRN) Scheduling

- Bu algoritmada tüm mevcut process'lerin cevap oranını (response ratio)
- bulmak ve en yüksek Cevap Oranı olanını seçmek gerekmektedir.
- Bir kez seçildiğinde bir process tamamlanana kadar CPU'da çalışacaktır.
- Bu zamanlama algoritması **Non-Preemptive** çalışır.

- Kriter – Cevap Oranı (Response Ratio)

- Response Ratio = (W + S)/S
- W : waiting time (bekleme süresi)
- S : burst time (işletim süresi)

Highest Response Ratio Next (HRRN) Scheduling

- 5 Thread için kuyruğa varış süreleri ve işletim süreleri aşağıdaki tabloda verilmiştir. Buna göre Highest Response Ratio Next (HRRN) algoritmasını kullanılarak process'lerin Gantt şemalarını gerçekleştiriniz.

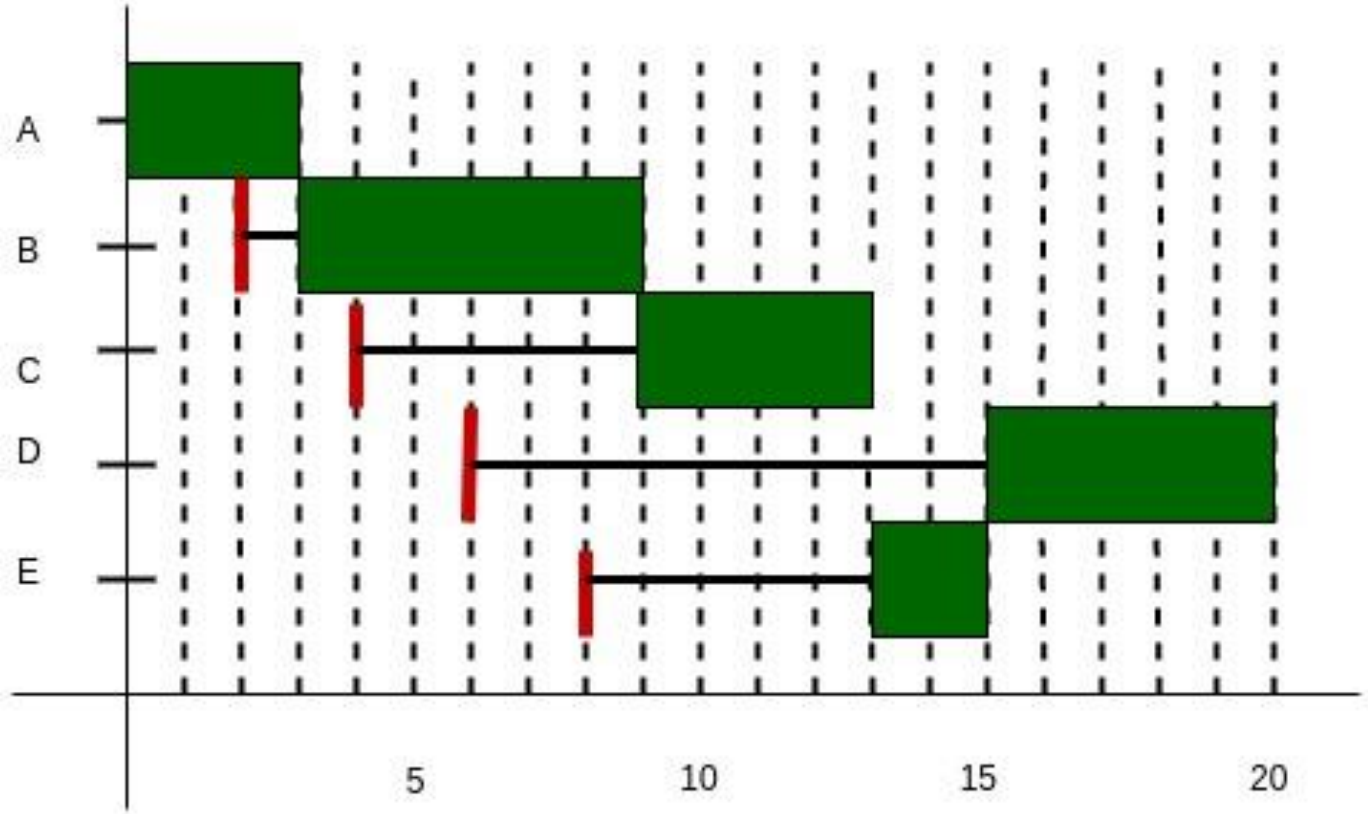
Thread	Arrival TIme	CPU Burst Length
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Highest Response Ratio Next (HRRN) Scheduling

- t = 0'da sadece tek bir thread var, yani A yürütülür.
- Aynı şekilde, t = 3'te sadece tek bir thread var, bu yüzden B yürütülür.
- Şimdi t = 9'da 3 adet thread mevcut (C, D ve E). C, D ve E sırasıyla 4, 6 ve 8 zamanlarından sonra kuyruğa girdiler. Bu nedenle,
- C bekleme süresi : 9 - 4 = 5
- D bekleme süresi : 9 - 6 = 3
- E bekleme süresi : 9 - 8 = 1
- C Response Ratio = (W + S)/S = 2,25
- D Response Ratio = (W + S)/S = 1,6
- E Response Ratio = (W + S)/S = 1,5
- Açıkça C en yüksek Cevap Oranına (Response Ratio) sahiptir ve bu yüzden C yürütülür.
- Sonraki t = 13'te 2 tane thread var (D ve E)
- D ve E'nin Cevap Oranları sırasıyla 2,4 ve 3,5'dir.
- Bundan sonra thread E seçilir ve son thread D seçilir.

Thread	Arrival TIme	CPU Burst Length
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Highest Response Ratio Next (HRRN) Scheduling

Thread	Arrival TIme	CPU Burst Length
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Gantt Chart -